

JAVA

Sintassi e costrutti di base



Il main program

L'inizio di un programma java è la funzione **main**

E' l'entry point
della **JVM**

```
public class Prova
{
    public static void main(String[] args)
    {
        istruzione
        istruzione
        ...
    }

    funzione1()

    funzione2()
}
```

Tutte le funzioni devono essere scritte all'interno della classe

Stile di codifica

- ◆ Spaziature: per separare gli elementi è necessario utilizzare almeno uno spazio
- ◆ Ogni istruzione finisce con ";"
- ◆ Si possono scrivere molte istruzioni nella stessa riga, ma è consigliabile una riga per ogni dichiarazione o istruzione.

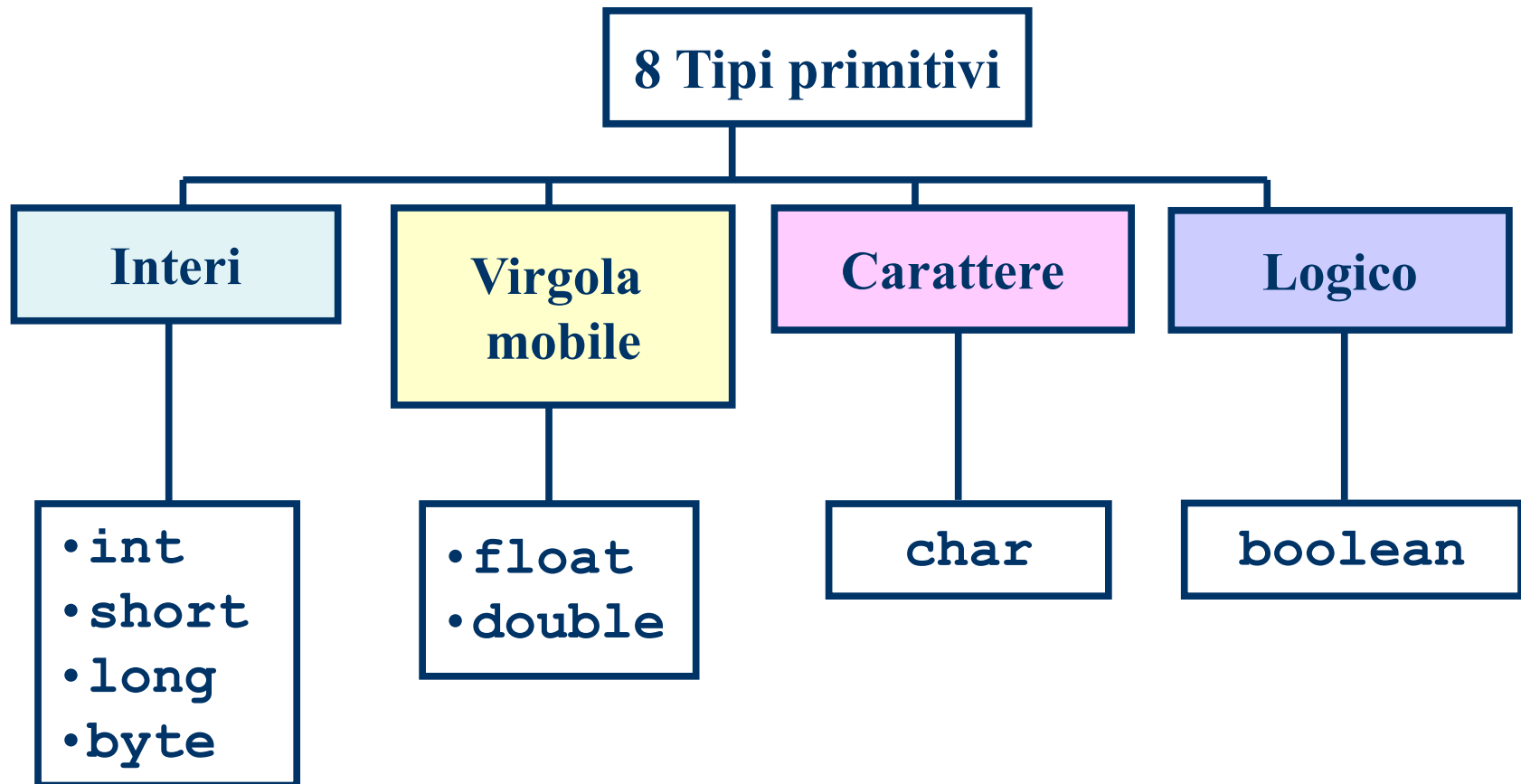
Commenti

Esistono tre diversi stili:

- ◆ `//` per commenti di una riga
- ◆ `/*` blocco di commenti
(più righe) `*/`
- ◆ `/**` Commenti javadoc:
 - `*` invocando uno speciale tool (javadoc.exe)
 - `*` sul codice commentato con questo stile
 - `*` viene prodotta una documentazione in HTML con
 - `*` formattazione standard della Sun
 - `*`
 - `*` Si utilizzano inoltre speciali tag per definire
 - `*` caratteristiche delle classi e dei metodi
 - `*` `/`

Tipi primitivi

Sono i tipi definiti nel linguaggio e dai quali si può partire con la costruzione di espressioni



Tipi Interi

Tipo	Requisiti di memorizzazione	Intervallo (inclusivo)
int	4 byte	Da -2.147.483.648 a + 2.147.483.647
short	2 byte	Da -32.768 a + 32.767
long	8 byte	- 9.223.372.036.854.775.808L a + 9.223.372.036.854.775.807L
byte	1 byte	Da - 128 a + 127

Tipi in virgola mobile

Tipo	Requisiti di memorizzazione	Intervallo (inclusivo)
float	4 byte	circa $\pm 3.40282347\text{E}+38$ F (6-7 cifre decimali significative)
double	8 byte	circa $\pm 1.79769313486231570\text{E}+308$ (15 cifre decimali significative)

Tipo carattere

- ◆ E' un carattere dello schema **Unicode** (raccolge tutti i caratteri esistenti), si rappresenta su 2 bytes.
- ◆ Si usa il delimitatore: singolo apice.
`char c1 = 'A' ;`
`char c2 = 'a' ;`
`char nullo = '\0' ;`
`char nullo = '\u0000' ;`
- ◆ Esistono anche delle sequenze di escape

Sequenza di escape	Nome	Valore Unicode
<code>\b</code>	backspace	<code>\u0008</code>
<code>\t</code>	tabulazione	<code>\u0009</code>
<code>\n</code>	nuova riga	<code>\u000a</code>
<code>\r</code>	ritorno carrello	<code>\u000d</code>
<code>\"</code>	virgolette doppie	<code>\u0022</code>
<code>\'</code>	virgolette singole	<code>\u0027</code>
<code>\\</code>	back slash	<code>\u005c</code>

Tipo logico

- ◆ Assume soltanto due valori: **true** o **false**
- ◆ Non si usano apici, né virgolette
- ◆ Esempio:

```
boolean flag = false;
```

- ◆ Si usano nelle istruzioni di controllo di flusso (if, while, do)
- ◆ Gli operatori comparativi restituiscono valori booleani.

Dichiarazione variabili

- ♦ Tutte le variabili devono essere dichiarate
- ♦ Una variabile viene definita da:
 - da un tipo
 - un identificatore

Regole degli identificatori

- ♦ sono case-sensitive
- ♦ possono contenere lettere, numeri, _, \$
- ♦ non possono iniziare con un numero

Convenzioni degli identificatori

- ♦ Inizia sempre con la lettera minuscola
- ♦ Se ci sono più parole si alterna minuscolo-maiuscolo
Es: `int maxValoreCalcolato;`

Blocchi di istruzioni

- Le parentesi graffe delimitano blocchi di istruzioni
- Racchiudono classi, metodi, loop, ecc

Non è possibile utilizzare lo **stesso nome** per variabili in blocchi annidati:

NB: In altri linguaggi di programmazione è permesso

```
{  
    int a =10;  
    {  
        int a =20; //NO!  
    }  
}
```

Scope di una variabile

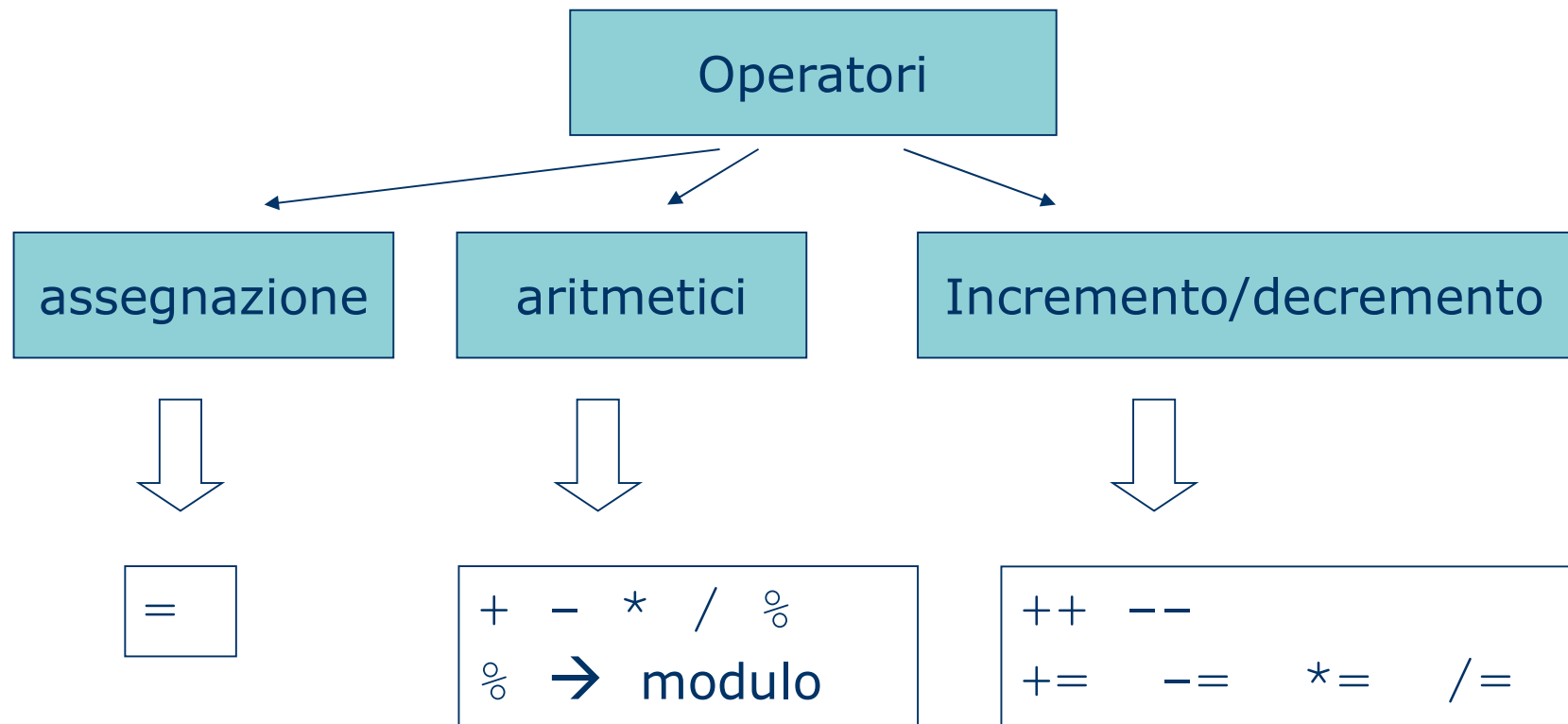
- ◆ Lo spazio di esistenza (e di visibilità) di una variabile è il suo **SCOPE**
- ◆ Oltre lo scope di una variabile, la memoria viene deallocata.

Scope di a

```
{  
    statement 1;  
    statement 2;  
    int a;  
    {  
        statement 3;  
        statement 4;  
    }  
    statement 5;  
    a = 10; //OK!  
    statement 6;  
}
```

System.out.println(a); // Errore: la variabile a non esiste più.

Operatori



Attenzione alla divisione. Il risultato

- è un intero, se gli operandi sono interi
- è un decimale, se almeno uno è con la virgola

Promozione automatica

- ◆ Nelle operazioni tra tipi diversi, il risultato **promosso al tipo più grande (preciso)** tra gli operandi

`int + byte → int`

`float + short → float`

`int + double → double`

Scala di precisione è:

`byte < short < int < long < float < double`

Invece se gli operandi sono dello stesso tipo:

`byte + byte → int`

`short + short → int`

`int + int → int`

`long + long → long`

`float + float → float`

`double + double → double`

Cast

```
int a;  
int b = 10;  
a = b;  
-----
```

```
int a = 4;  
short b = 10;  
a = b;  
-----
```

```
byte a;  
int b = 10;  
a = b; // NO!!
```

```
a = (byte)b;  
// SI! Ma i rischi  
// sono consapevoli
```

Quanto vale a in questi esempi?

Questa istruzione è potenzialmente pericolosa. Infatti il compilatore dà errore

Java impone al programmatore di fare scelte consapevoli.

Cast

- ♦ Il cast è una forzatura locale di tipo
- ♦ La sintassi del cast prevede l'indicazione del tipo di destinazione tra () e va indicato prima della variabile su cui fare il cast

♦ Es.

```
int x = 10;
```

```
byte y = (byte) x;
```

Esempio

Esempio di promozione:

```
byte a = 18;
```

```
byte b = 10;
```

```
byte c = a+b;
```

```
// errato perché a + b viene di tipo int
```

```
int c = a + b;
```

```
// giusto oppure byte c = (byte) (a + b);
```

Operatori relazionali

- ◆ Permettono il confronto tra due valori che stabiliscono relazioni di uguaglianza e d'ordine.

Operatore	Significato
==	uguale a
!=	diverso da
>	maggiore
<	minore
>=	maggiore o uguale
<=	minore o uguale

Operatori relazionali

- ♦ Java usa simboli diversi per le assegnazioni e per la verifica dell'uguaglianza.
- ♦ `(a == 7)` darà **true** se a vale 7
false diversamente
- ♦ `a = 7` assegna 7 alla variabile a

NOTA:

```
if(a = 7) // NON compila!!
```

Operatori booleani

Operatore	Significato
 	OR sc
&&	AND sc
!	NOT
^	XOR
 	OR
&	AND

sc → ShortCircuited,
cioè ottimizza controllo delle
condizioni in OR e in AND

Esempio:

Supponiamo che

```
int a = 7;
```

```
int b = 10;
```

Allora

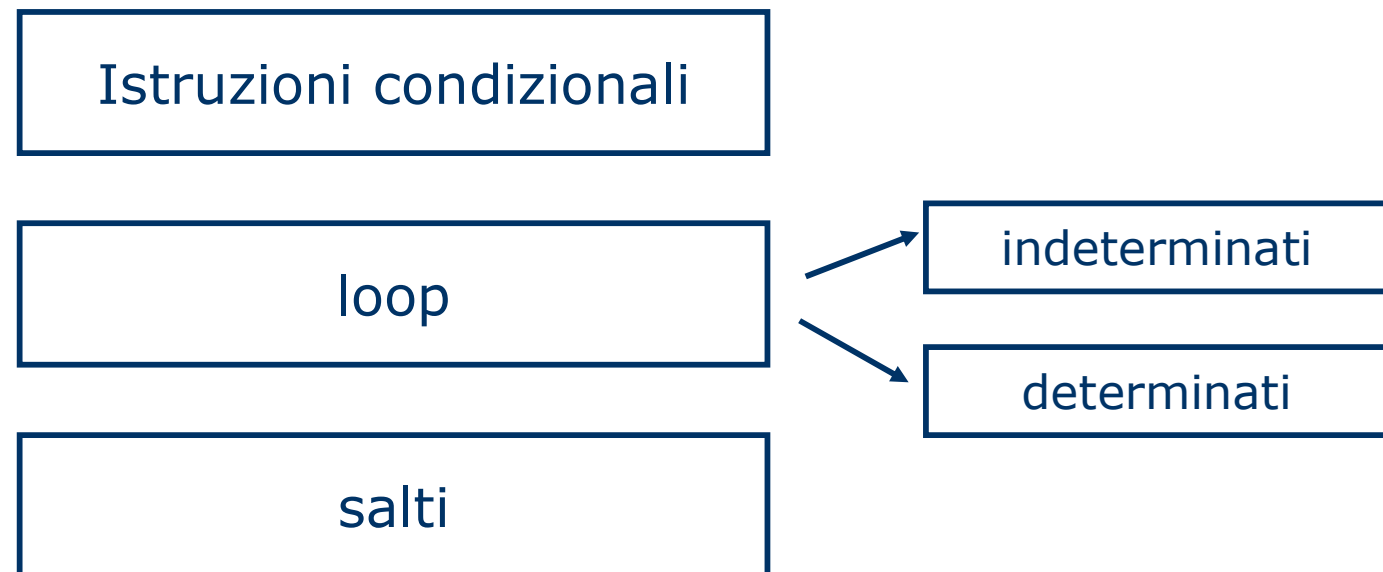
(a==7) && (b>0) → *true*

(a==7) || (b>0) → *true*

!(a<4) → *true*

(a==7) ^ (b>0) → *false*

Controllo di flusso



Istruzione if-else

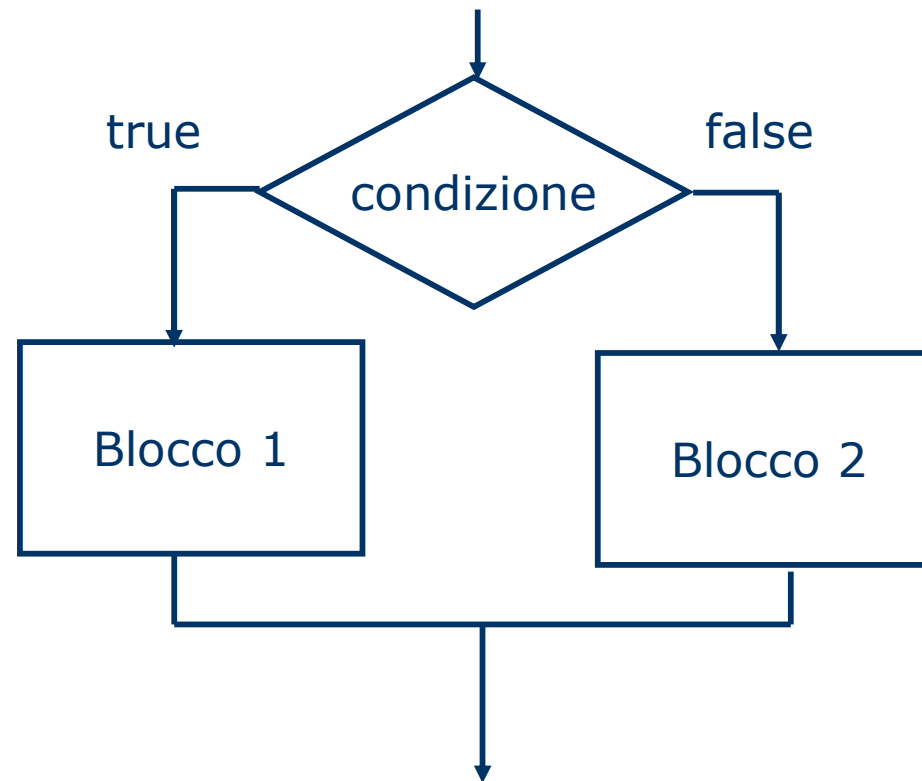
```
if (condizione)
{
    istruzione1;
    istruzione2;
    ...
}
```

oppure

```
if (condizione)
    istruzione;
```

- La condizione deve essere sempre tra parentesi.
- Se c'è una sola istruzione → le parentesi graffe possono essere omesse
- Il blocco **else** è opzionale
- Se voglio più condizioni posso usare anche **else if**

Istruzione if-else



Istruzione switch

E' il dispositivo di condizione multipla che consente di analizzare il valore di una **variabile** rispetto ad un set di **costanti**

La variabile (e le costanti) possono essere solo dei seguenti tipi:

- **char**
- **byte**, **short**, **int**
- **enum** (da java7)
- **String** (da java7)

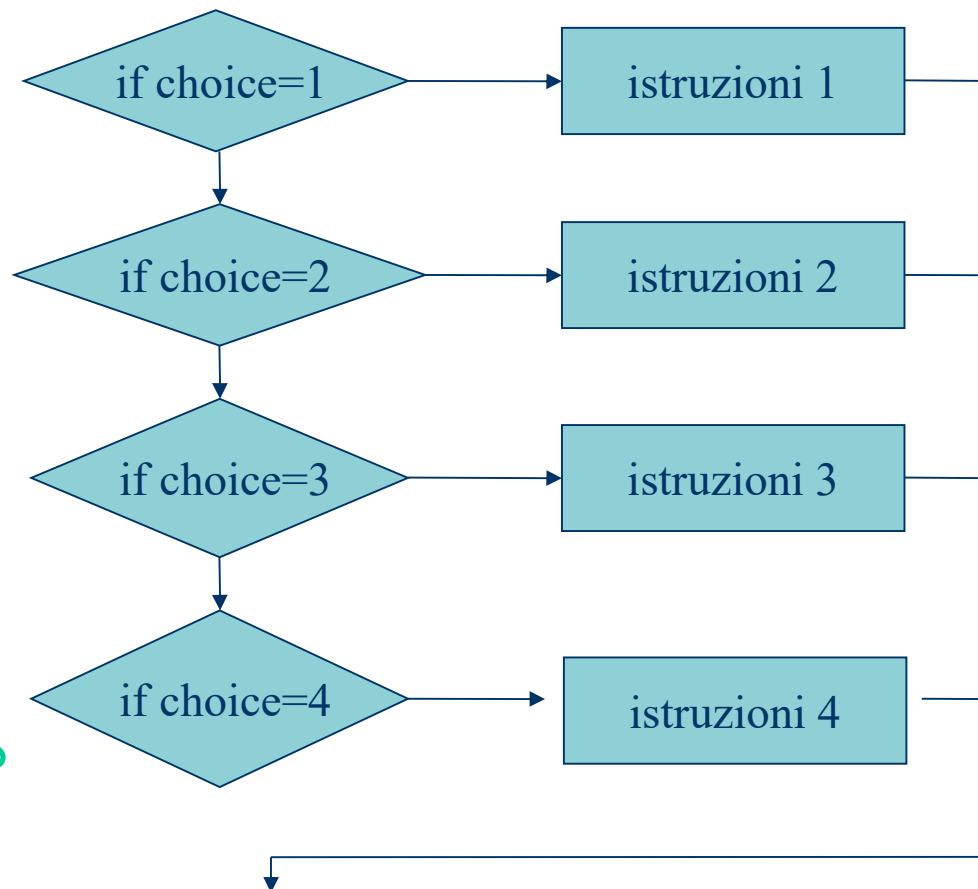
Non funziona con

- **long**
- Tutti i tipi floating point
- intervalli di valori

'classic' switch

Se choice è un `int` letto da tastiera

```
switch(choice)
{
    case 1:
        istruz1;
        break;
    case 2:
        istruz2;
        break;
    case 3:
        istruz3;
        break;
    case 4 :
        istruz4;
        break;
    default:
        // input errato
        ...
}
```



L'istruzione break

- ♦ L'istruzione **break** è opzionale e serve per uscire dallo **switch**
- ♦ Se non si inserisce nel case, accade che eseguito un case si «cade» nel case sottostante (successivo)
- ♦ Solitamente ogni case rappresenta un caso indipendente e quindi serve usare **break**.
- ♦ Se però voglio eseguire lo stesso codice per 2 o più case distinti, potrei ottimizzare il codice raggruppando i case in questione e usando un solo **break** (→ segue esempio)

Esempio break

```
switch(choice)
{
    case 1:
    case 2:
        istruzioni;
        break;
    case 3:
        istruz3;
        break;
    default:
        // input errato
        ...
}
```



Se choice vale 1, entra nel case 1 e cade nel case successivo. Quindi eseguirà le istruzioni del case 2 e infine uscirà dallo switch

Da java 12, valore di ritorno

- ♦ Lo *switch statement* diventa *un'espressione*, quindi si può utilizzare la keyword **break** seguita dal valore da tornare

```
String d = switch (day) {  
    case "Monday":  
        break "Weekday";  
    case "Tuesday":  
        break "Weekday";  
    case "Wednesday":  
        break "Weekday";  
    case "Thursday":  
        break "Weekday";  
    case "Friday":  
        break "Weekday";  
    case "Saturday":  
        break "Weekend";  
    case "Sunday":  
        break "Weekend";  
    default: break "unknown";  
};
```

Supponiamo che **day** sia una stringa.

NB: Siccome il ritorno dello switch valorizza una variabile è importante prevedere il caso di **default**

Da java 12, case multipli

- ◆ Lo switch può gestire case multipli
- ◆ Si utilizza la virgola per separare le costanti

```
String s = switch (day) {  
    case "Monday", "Tuesday", "Wednesday", "Thursday", "Friday":  
        break "Weekday";  
    case "Saturday", "Sunday":  
        break "Weekend";  
};
```

Da java 12, uso di arrow

- ◆ Viene introdotto l'operatore arrow (freccia, già usata in java 8 per le lambda expression)
- ◆ L'operatore arrow -> sostituisce il ':' e il 'break'

```
String s = switch (day) {  
    case "Monday"      -> "Weekday";  
    case "Tuesday"     -> "Weekday";  
    case "Wednesday"   -> "Weekday";  
    case "Thursday"    -> "Weekday";  
    case "Friday"       -> "Weekday";  
    case "Saturday"    -> "Weekend";  
    case "Sunday"       -> "Weekend";  
    default             -> "unknown";  
};
```

NB: fino a java 13, tutte queste nuove features si potevano abilitare o meno, da **java 14 diventano permanenti**

Loop indeterminati

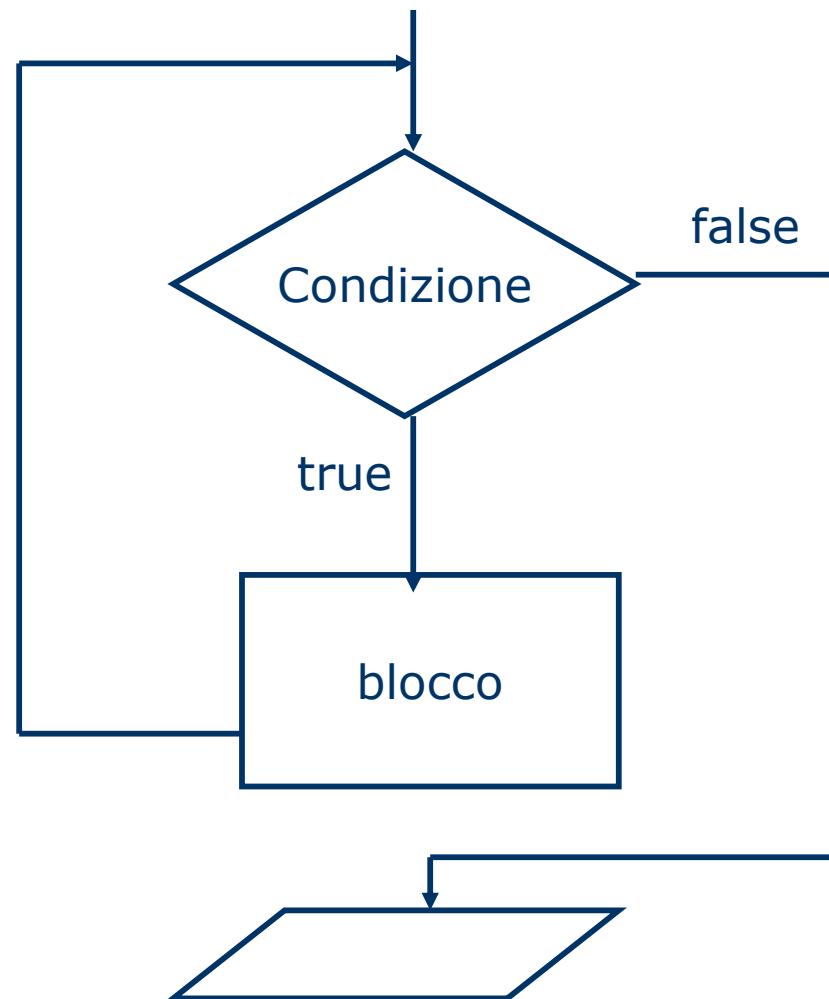
Il loop **while** esegue una verifica iniziale.

```
while (condizione)
{
    blocco istruzioni
}
```

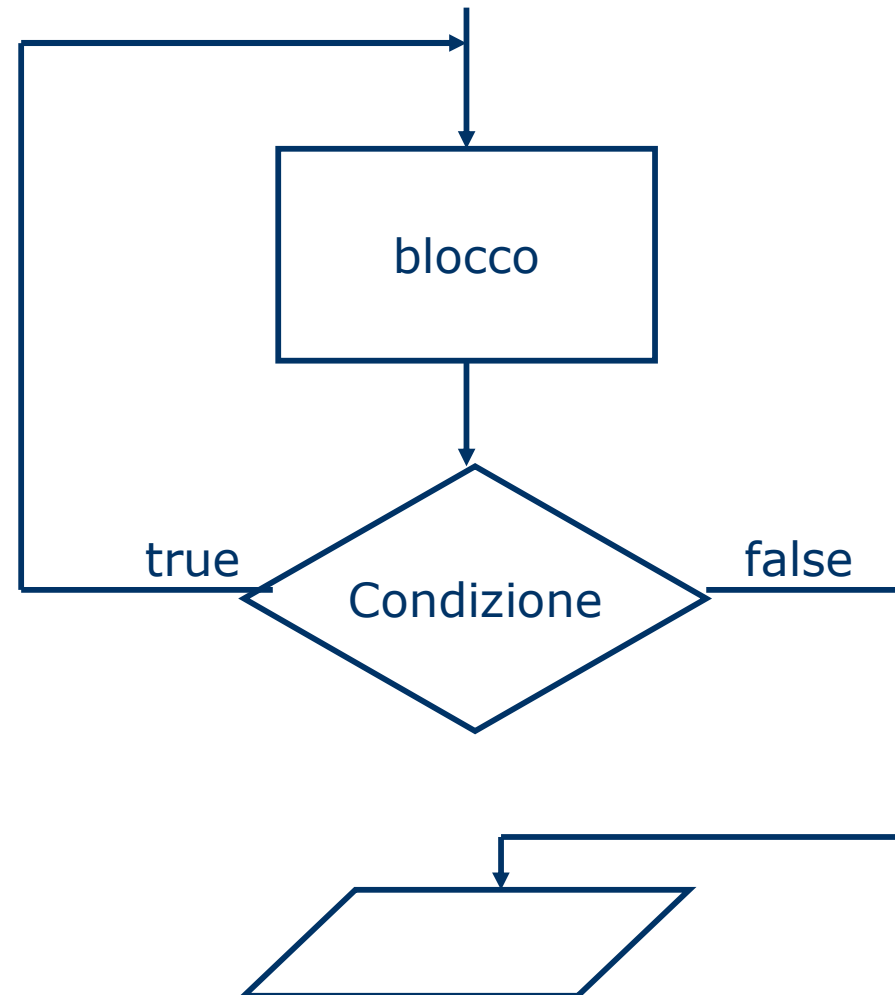
- ♦ Se la condizione iniziale è `false`, non viene eseguito
- ♦ Il loop while esegue il blocco finché la condizione iniziale è `true`
- ♦ Per eseguire almeno una volta il blocco occorre usare il `do-while` :

```
do
{
    blocco istruzioni
}
while (condizione);
```

Il while



Il do - while



Loop determinati

```
for ( inizio; iterazione; passo)
{
    istruzione1;
    istruzione2;
    istruzione3;
}
```

Il ciclo si ripete un **numero prefissato** di volte, secondo i parametri dati da:

- **inizio ciclo**
- **iterazione del ciclo**
- **passo**

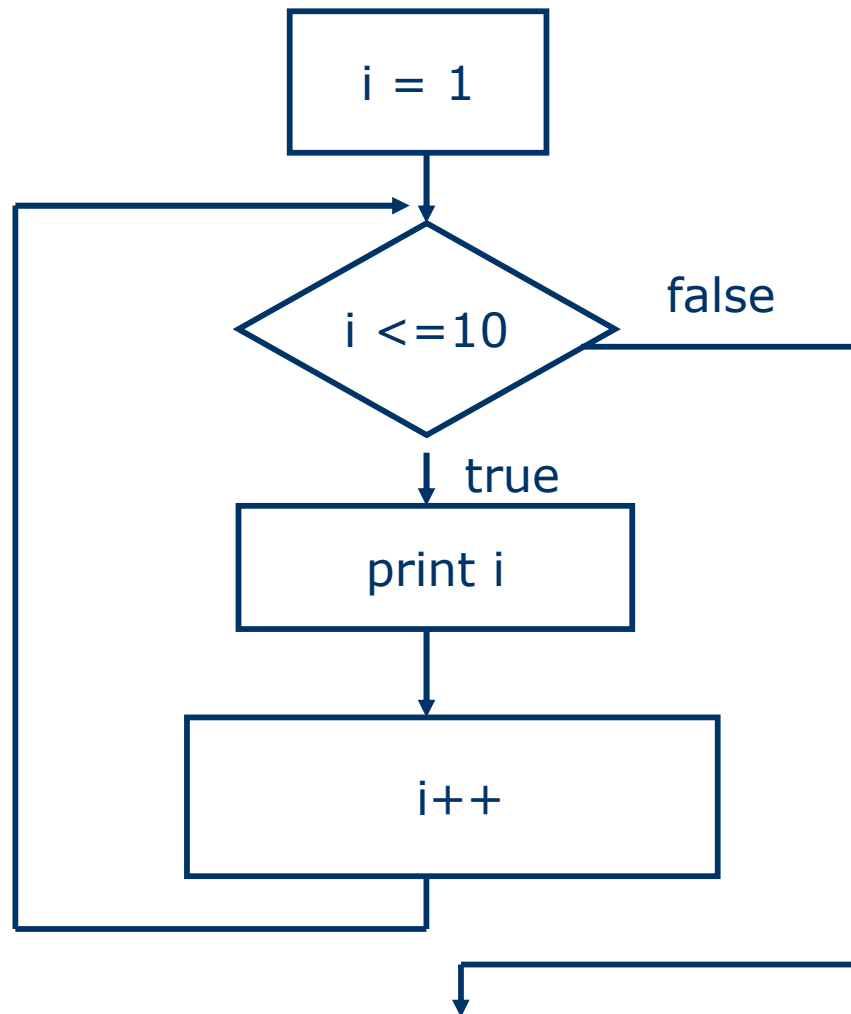
La variabile per ciclare si può dichiarare dentro le parentesi tonde

Loop determinati

```
for (int i = 1; i <=10; i++)  
{  
    System.out.println(i);  
}
```

Stampa i
numeri
da 1 a 10

1
2
3
4
5
6
7
8
9
10



continue

- ◆ Consente di procedere all'iterazione successiva del ciclo dove compare (eseguendo una sorta di *salto*)
- ◆ Per effettuare un salto di 2 cicli, è necessario il costrutto `label-continue`

```
for(int i =0; i< 20; i++){  
    for(int j =0; j< 10; j++){  
        if(i==12 && j==5)  
            continue;  
        System.out.println(j) ;  
    }  
    System.out.println("  --  "+i) ;  
}
```

continue

In corrispondenza di $i=12$ e $j=5$ non viene effettuata la stampa di $j = 5$ e va all'iterazione successiva $j = 6$

label-continue

- ◆ Attraverso un'etichetta posso indentificare un loop e eseguire il continue al loop etichettato
 - L'etichetta deve avere un nome composto da una sola parola e deve essere seguito da ":"

label

continue

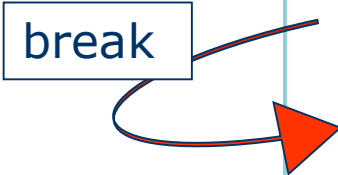
```
etichetta:
for(int i =0; i< 20; i++){
    for(int j =0; j< 10; j++){
        if(i==12 && j==5)
            continue etichetta;
        System.out.println(j);
    }
    System.out.println(" -- "+i);
}
```

In corrispondenza di i=12 e j=5 si salta all'iterazione con i=13: non viene effettuata la stampa di j 5...9 e la stampa di i con i=12

break

- ♦ L'istruzione **break** consente di interrompere il loop dove compare

```
for(int i =0; i< 20; i++){  
    for(int j =0; j< 10; j++){  
        if(i==12 && j==5)  
            break;  
        System.out.println(j);  
    }  
    System.out.println(" -- "+i);  
}
```



In corrispondenza di i=12 e j=5 il ciclo di j viene terminato!!! Non esegue le stampe di j 5...9 ma esegue la stampa di i = 12

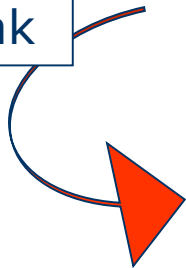
label-break

- ♦ Etichettando un loop posso eseguire il **break** al loop etichettato, ottenendo l'interruzione di quest'ultimo.

label

```
etichetta:
for(int i =0; i< 20; i++){
    for(int j =0; j< 10; j++){
        if(i==12 && j==5)
            break etichetta;
        System.out.println(j);
    }
    System.out.println(" -- "+i);
}
```

break



In corrispondenza di i=12 e j=5 il ciclo su i viene terminato (e ovviamente anche quello interno su j)

Passaggio parametri

- ◆ Per tutti i tipi **primitivi** il passaggio dei parametri è **per valore**.
- ◆ Il metodo esegue una copia del parametro
- ◆ Le modifiche sulla copia non sono visibili al chiamante

Esempio

```
public class PassaggioPerValore
{
    public static void main (String[] args) {
        int n = 30;
        System.out.println("n vale " + n); // Stampa 30
        nonModifica(n);
        System.out.println("n vale ancora " + n); // Stampa 30
    }

    public static void nonModifica(int i) {
        System.out.println("i vale " + i); // Stampa 30
        i = 0;
        System.out.println("adesso i vale " + i); // Stampa 0
    }
}
```

Stampe e formattazioni

- ◆ Per stampare numeri con formattazione si usa la funzione `printf` invece della `println`
- ◆ La funzione prende in input:
 - Una stringa di formattazione
 - Il valore da formattare
- ◆ *Le stringhe di formattazione sono fissate e sono moltissime, per venire incontro a tutte le esigenze*



Appendice

STAMPE E FORMATTAZIONI

Formattare i decimali

- ◆ Le stringhe di formattazione più usate sono:
 - `%f` → floating point
 - `%g` → general
- ◆ La precisione decimale si indica tra il carattere `%` e la lettera (`f` oppure `g`) e indica:
 - Numero cifre intere
 - Numero cifre decimali
 - Numero cifre totali di rappresentazioni

Esempi

```
double x = 12.345;  
// stampa senza formattazione  
System.out.println(x);  
// stampa con formattazione GENERAL  
System.out.printf("%3.3g", x);  
// stampa con formattazione FLOATING POINT  
System.out.printf("%2.2f", x);
```

Stamperà:

12.345 no formattazione

12.3 fmt con 3 cifre intere e 3 cifre totali di stampa

12,35 fmt con 2 cifre intere e 2 posti decimali(round)

Tutorial



Sul sito della Oracle c'è una guida pratica per programmatori, ricca di esempi e discussioni su specifici argomenti.

<http://download.oracle.com/javase/tutorial/>

Trails Covering the Basics

[Getting started](#)

[Learning java language](#)

[Essential java classes](#)

[Collection](#)

[Swing](#)

[Deployment](#)

[Certification](#)